

DVWA WITH DOCKER REPORT / REMEDIATION.

1. A01:2021-Broken Access Control (Insecure Direct Object References)

- **DVWA Module:**
- **The Vulnerability:** The application assumes the user has the authorization to request a file or change a password simply because they know the URL path or parameter.
- **How it looks in DVWA:** Modifying the page parameter in File Inclusion (page=../../../../../etc/passwd) allows unauthorized viewing of system files.
- **Remediation Steps:**
 1. **Implement an Access Control List (ACL):** Validate that the logged-in session possesses the specific role required to view the resource.
 2. **Use Whitelisting for File Inclusion:** Do not pass user input directly into file paths. Use a hardcoded whitelist array in PHP:

php

```
$allowed_pages = ['main.php', 'help.php', 'contact.php'];  
if (!in_array($page, $allowed_pages)) { die("Access Denied"); }
```

2. A02:2021-Cryptographic Failures (Sensitive Data Exposure)

- **DVWA Module:** *Brute Force / View Source*
- **The Vulnerability:** DVWA transmits credentials in cleartext over HTTP (as seen in your previous network capture analysis) and stores user passwords using weak or no hashing (e.g., MD5).
- **How it looks in DVWA:** Inspecting the database config or a database dump reveals passwords stored in raw MD5, which can be instantly cracked via rainbow tables.
- **Remediation Steps:**
 1. **Enforce HTTPS:** Modify your Docker/Apache configuration to redirect traffic from port 80 to port 443 using TLS 1.3.
 2. **Use Strong Hashing Algorithms:** Replace legacy MD5 hashes with PHP's native password_hash() function, which defaults to **Argon2** or **Bcrypt**:

php

```
$secure_password = password_hash($user_password, PASSWORD_DEFAULT);
```

3. A03:2021-Injection (SQL Injection & Command Injection)

- **DVWA Module:** *SQL Injection / Command Injection*

- **The Vulnerability:** User-supplied inputs are directly concatenated into backend SQL queries or system commands without sanitization.
- **How it looks in DVWA:**
 - Command Injection input: 127.0.0.1 && cat /etc/passwd
 - SQL Injection input: %37' OR '1'='1

- **Remediation Steps:**

1. **For SQLi (Use Prepared Statements):** Never concatenate variables into SQL strings. Use PDO with parameterized queries:

php

```
$stmt = $pdo->prepare('SELECT first_name, last_name FROM users WHERE user_id = :id');
```

```
$stmt->execute(['id' => $id]);
```

```
$user = $stmt->fetch();
```

2. **For Command Injection (Avoid System Calls):** If you must use them, strictly escape arguments using `escapeshellarg()` or structure the inputs using built-in language APIs rather than shell wrappers.

4. A04:2021-Insecure Design

- **DVWA Module:** *Weak Session IDs / Insecure CAPTCHA*
- **The Vulnerability:** The security flaws lie in the business logic and design architecture itself, such as incrementing session IDs sequentially (e.g., 1, 2, 3) rather than using a cryptographically secure random value.
- **How it looks in DVWA:** Generating multiple session tokens shows a predictable pattern, allowing an attacker to guess another user's session identifier.
- **Remediation Steps:**
 1. **Leverage Secure Native Sessions:** Do not design custom session generation mechanisms. Rely on PHP's default session management (`session_start()`), which generates strong, random tracking tokens.

5. A05:2021-Security Misconfiguration

- **DVWA Module:** *Setup / PHP Info*
- **The Vulnerability:** The underlying infrastructure (Docker container, Apache server, PHP engine) exposes verbose error logs, default configurations, or debug pages to the end-user.
- **How it looks in DVWA:** Forcing an unhandled exception or visiting `phpinfo.php` displays your internal file paths, module versions, and environment variables.
- **Remediation Steps:**
 1. **Disable Error Displaying:** Modify the `php.ini` file inside your Docker container to suppress frontend error reporting:

ini

display_errors = Off

log_errors = On

Strip Server Tokens: Update your Apache configuration (httpd.conf) to minimize information disclosure headers:

apache

ServerTokens Prod

ServerSignature Off.

6. A06:2021-Vulnerable and Outdated Components

- **DVWA Module:** *Application Environment*
- **The Vulnerability:** DVWA relies on legacy web server versions, unpatched PHP instances, or outdated JavaScript libraries (like old versions of jQuery) built into the repository.
- **How it looks in DVWA:** Running a vulnerability scanner like Nikto against your DVWA Docker container reveals old CVEs associated with the operating system or active Apache version.
- **Remediation Steps:**
 1. **Update the Docker Base Image:** Regularly rebuild or pull updated base layers in your Dockerfile (e.g., upgrading from php:5.6-apache to php:8.2-apache).
 2. **Dependency Scanning:** Integrate Software Composition Analysis (SCA) tools into your delivery pipeline to scan third-party dependencies.

7. A07:2021-Identification and Authentication Failures

- **DVWA Module:** *Brute Force*
- **The Vulnerability:** The application lacks defense mechanisms against automated credential attacks, dictionary attacks, or password spraying.
- **How it looks in DVWA:** Running Hydra or Burp Suite Intruder allows an investigator to try millions of password combinations against the login panel without facing account locks or delays.
- **Remediation Steps:**
 1. **Implement Rate Limiting / Account Lockout:** Track failed login attempts by IP or username. Lock the account or enforce an exponential backoff delay after 5 failed attempts.
 2. **Enforce Multi-Factor Authentication (MFA):** Require a secondary time-based token (TOTP) for administrative portals.

8. A08:2021-Software and Data Integrity Failures

- **DVWA Module:** *File Upload*

- **The Vulnerability:** Accepting uploaded files without validating their true content or signature, allowing an attacker to overwrite sensitive system execution targets or upload a web shell.
- **How it looks in DVWA:** Uploading a file named shell.php allows an attacker to navigate directly to the uploaded directory and execute remote commands on the container.
- **Remediation Steps:**
 1. **Rename Uploaded Files:** Generate a cryptographically secure random string (like a UUID) to serve as the new file name, stripping the original user-supplied name.
 2. **Validate MIME-Types & Extensions:** Check the actual file bytes (magic bytes), enforce strict extension whitelisting, and store uploads outside the public web root directory (/var/www/html).

9. A09:2021-Security Logging and Monitoring Failures

- **DVWA Module:** *Global Application Behavior*
- **The Vulnerability:** Auditable security events (such as failed authentication attempts, privilege elevations, or input validation errors) are either not logged at all or stored locally where an attacker can clear them.
- **How it looks in DVWA:** Brute-forcing the web login leaves no trace or structured alerts within a centralized dashboard for real-time detection.
- **Remediation Steps:**
 1. **Centralize Docker Logs:** Use Docker log drivers to stream events out of the container to a centralized syslog server or SIEM platform.
 2. **Structured Application Logging:** Implement a library like Monolog in your PHP code to record failed events in a tamper-resistant format.

10. A10:2021-Server-Side Request Forgery (SSRF)

- **DVWA Module:** *XSS / File Inclusion (Advanced combinations)*
- **The Vulnerability:** The web application fetches a remote resource based on a user-supplied URL without validating the destination address, allowing attackers to coerce the server into scanning internal network interfaces.
- **How it looks in DVWA:** Forcing the application to fetch a file from http://169.254.169 to extract cloud infrastructure metadata credentials.
- **Remediation Steps:**
 1. **Restrict Network Access:** Implement strict network firewall rules on your Docker container sandbox, blocking it from initiating outbound requests to internal subnets or loopback adapters (127.0.0.1).
 2. **URL Whitelisting:** If the application must fetch remote assets, restrict user inputs to a strict whitelist of approved external host domains.